

TP2 NoSQL : Gestion des Rôles

RBAC et Création d'Utilisateurs - Partie 2

Travail Pratique N°2 - Module NoSQL



Saad KORCHI

Étudiant en Cybersécurité

Encadré par :

Pr. AMAMOU Ahmed

EIDIA Cybersécurité

SOMMAIRE

1	Introduction	3
2	Gestion des rôles (RBAC)	3
2.1	Création du rôle	3
2.2	Explication détaillée des privilèges	3
2.3	Pourquoi créer un rôle personnalisé ?	4
2.4	Structure des privilèges MongoDB	4
3	Création des utilisateurs	4
3.1	Utilisateur applicatif - soc_app	4
3.2	Utilisateur lecture seule - soc_analyst_ro	4
3.3	Manager incidents - soc_manager	5
3.4	Tableau récapitulatif des utilisateurs	5
4	Connexion backend sécurisée	5
4.1	Fichier .env	5
4.2	Détail de l'URI de connexion	5
4.3	Avantages de cette approche	6
4.4	Vérification de la connexion	6
	Captures d'écran	7

1 Introduction

Contexte - Gestion des rôles et utilisateurs

Cette deuxième partie du projet SOC-lite se concentre sur la mise en place du **contrôle d'accès basé sur les rôles (RBAC)**. Après avoir sécurisé l'accès à MongoDB par l'authentification, nous devons maintenant définir précisément ce que chaque utilisateur peut faire.

Le RBAC (Role-Based Access Control) est un modèle de contrôle d'accès où les permissions sont attribuées à des rôles, et les utilisateurs se voient assigner des rôles. Cette approche présente plusieurs avantages :

- **Principe du moindre privilège** : Chaque utilisateur dispose uniquement des droits nécessaires à ses fonctions
- **Gestion simplifiée** : Les droits sont gérés par rôle, pas individuellement
- **Audit facilité** : Traçabilité claire des actions par rôle

2 Gestion des rôles (RBAC)

Un rôle personnalisé a été créé pour gérer les incidents. Ce rôle spécifique, nommé `incidentManager`, permet de séparer clairement les responsabilités entre les différents acteurs du SOC.

2.1 Création du rôle

```
1 db.createRole({
2   role: "incidentManager",
3   privileges: [
4     {
5       resource: { db: "soc_lite", collection: "incidents" },
6       actions: ["find", "insert", "update", "remove"]
7     }
8   ],
9   roles: []
10 })
```

2.2 Explication détaillée des privilèges

- **find** : Permet de rechercher et consulter les incidents. Cette action est essentielle pour l'analyse des alertes.
- **insert** : Autorise la création de nouveaux incidents. Un analyste peut ainsi ouvrir un ticket lorsqu'une alerte est confirmée.
- **update** : Permet la mise à jour des incidents existants (changement de statut, ajout de commentaires, résolution).

- **remove** : Autorise la suppression des incidents. Cette action est généralement réservée aux rôles avancés (nettoyage des faux positifs).

2.3 Pourquoi créer un rôle personnalisé ?

MongoDB fournit des rôles intégrés comme **readWrite**, mais ceux-ci sont trop permissifs dans un contexte de sécurité. Le rôle **readWrite** donnerait accès à **toutes** les collections de la base, y compris **security_logs** et **users**. Notre rôle **incidentManager** restreint l'accès exclusivement à la collection **incidents**, conformément au principe de séparation des tâches.

2.4 Structure des privilèges MongoDB

Dans MongoDB, un privilège est défini par :

- **resource** : La cible du privilège (base de données, collection, cluster)
- **actions** : Les opérations autorisées (CRUD, index management, etc.)

Cette granularité fine permet une sécurité extrêmement précise, adaptée aux exigences d'un SOC.

3 Création des utilisateurs

Trois types d'utilisateurs ont été créés pour répondre aux différents besoins du SOC. Chaque utilisateur se voit attribuer un rôle spécifique qui détermine ses permissions.

3.1 Utilisateur applicatif - soc_app

```
1 db.createUser({
2   user: "soc_app",
3   pwd: "AppP@ss2026!",
4   roles: [{ role: "readWrite", db: "soc_lite" }]
5 })
```

Rôle et permissions :

- Rôle : **readWrite** sur la base **soc_lite**
- Actions autorisées : Lecture et écriture sur toutes les collections
- Utilisation : Connecte le backend Node.js à la base de données

3.2 Utilisateur lecture seule - soc_analyst_ro

```
1 db.createUser({
2   user: "soc_analyst_ro",
3   pwd: "AnalystR0@2026!",
4   roles: [{ role: "read", db: "soc_lite" }]
5 })
```

Rôle et permissions :

- Rôle : `read` sur la base `soc_lite`
- Actions autorisées : Lecture uniquement sur toutes les collections
- Utilisation : Comptes d'analystes juniors ou tableaux de bord en lecture seule

3.3 Manager incidents - `soc_manager`

```

1 db.createUser({
2   user: "soc_manager",
3   pwd: "Mgr$ecure2026!",
4   roles: [{ role: "incidentManager", db: "soc_lite" }]
5 })

```

Rôle et permissions :

- Rôle : `incidentManager` (personnalisé)
- Actions autorisées : CRUD complet uniquement sur la collection `incidents`
- Utilisation : Gestion avancée des incidents par les managers SOC

3.4 Tableau récapitulatif des utilisateurs

Utilisateur	Rôle	Permissions
<code>soc_app</code>	<code>readWrite</code>	Lecture/écriture complète sur <code>soc_lite</code>
<code>soc_analyst_ro</code>	<code>read</code>	Lecture seule sur <code>soc_lite</code>
<code>soc_manager</code>	<code>incidentManager</code>	CRUD uniquement sur <code>incidents</code>

4 Connexion backend sécurisée

Le backend Node.js se connecte à MongoDB avec un utilisateur dédié, distinct du compte administrateur. Cette ségrégation des comptes est une pratique essentielle de sécurité.

4.1 Fichier `.env`

```

1 MONGO_URI=mongodb://soc_app:AppP@ss2026!@127.0.0.1:27017/soc_lite?
  authSource=soc_lite

```

4.2 Détail de l'URI de connexion

- `mongodb://` : Protocole de connexion MongoDB
- `soc_app:AppP@ss2026!` : Identifiants de l'utilisateur applicatif
- `127.0.0.1:27017` : Hôte et port (localhost uniquement)
- `/soc_lite` : Base de données cible
- `?authSource=soc_lite` : Spécifie que les identifiants sont validés dans la base `soc_lite`

4.3 Avantages de cette approche

1. **Isolation des privilèges** : Le compte applicatif ne peut effectuer que les opérations nécessaires au fonctionnement normal
2. **Pas d'utilisation du compte admin** : Le compte adminSoc reste réservé aux opérations d'administration
3. **Sécurité renforcée** : En cas de compromission du backend, l'attaquant ne dispose que de droits limités
4. **Auditabilité** : Les actions du backend sont tracées avec le compte soc_app

4.4 Vérification de la connexion

Une fois le backend démarré, la console confirme la connexion réussie :

```
1 Server running on port 5000
2 MongoDB connected: 127.0.0.1
```

Fin du TP2 NoSQL - La suite sera traitée dans le TP3

Captures d'écran

```

soc_lite> db.getRoles()
{
  roles: [
    {
      _id: 'soc_lite.incidentManager',
      role: 'incidentManager',
      db: 'soc_lite',
      roles: [],
      isBuiltin: false,
      inheritedRoles: []
    }
  ],
  ok: 1
}
soc_lite>

```

FIGURE 1 – Figure 1

```

soc_lite> db.getUsers()
{
  users: [
    {
      _id: 'soc_lite.soc_analyst_ro',
      userId: UUID('00ffb4e8-c221-4ea3-b246-2bfda4a82111'),
      user: 'soc_analyst_ro',
      db: 'soc_lite',
      roles: [ { role: 'read', db: 'soc_lite' } ],
      mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
    },
    {
      _id: 'soc_lite.soc_app',
      userId: UUID('fbfbcbf4-1d64-4934-92b8-4ac7621f5bc1'),
      user: 'soc_app',
      db: 'soc_lite',
      roles: [ { role: 'readWrite', db: 'soc_lite' } ],
      mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
    },
    {
      _id: 'soc_lite.soc_manager',
      userId: UUID('47003629-a25b-479e-9306-b4dfdbda225c'),
      user: 'soc_manager',
      db: 'soc_lite',
      roles: [ { role: 'incidentManager', db: 'soc_lite' } ],
      mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
    }
  ],
  ok: 1
}
soc_lite>

```

FIGURE 2 – Figure 2

```

PS D:\4eme-année\application SOC-lite\backend> npm run dev

> soc-lite-backend@1.0.0 dev
> nodemon server.js
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
Server running on port 5000
MongoDB connected: 127.0.0.1

```

FIGURE 3 – Figure 3

```

already on db soc_lite
soc_lite> db.security_logs.insertOne({ event_type: "test_write", message: "tentative" })
MongoServerError[Unauthorized]: not authorized on soc_lite to execute command { insert: "security_logs", docum
ents: [ { event_type: "test_write", message: "tentative", _id: ObjectId('69d23e75ed56b795ce8563b1') } ], order
ed: true, lsid: { id: UUID("f4d1cb31-dee4-446e-8ee2-dea300f9c80a") }, $db: "soc_lite" }
soc_lite> db.incidents.deleteOne({ incident_id: "INC-2025-001" })
MongoServerError[Unauthorized]: not authorized on soc_lite to execute command { delete: "incidents", deletes:
[ { q: { incident_id: "INC-2025-001" }, limit: 1 } ], ordered: true, lsid: { id: UUID("f4d1cb31-dee4-446e-8ee2
-dea300f9c80a") }, $db: "soc_lite" }

```

FIGURE 4 – Figure 4

```

soc_lite> db.incidents.find()
[
  {
    _id: ObjectId('69d1b34f27551dd738edcd6a'),
    incident_id: 'INC-2025-001',
    title: 'Brute force SSH sur srv-web-01',
    description: 'Multiples tentatives SSH échouées depuis 192.168.1.105',
    severity: 'high',
    status: 'Investigating',
    assigned_to: 'alice_analyst',
    tags: [ 'brute-force', 'ssh' ],
    related_log_ids: [
      ObjectId('69d1b34f27551dd738edcd64'),
      ObjectId('69d1b34f27551dd738edcd65')
    ],
    comments: [
      {
        author: 'alice_analyst',
        text: 'Pattern brute-force confirmé.',
        timestamp: ISODate('2025-03-03T10:35:00.000Z')
      }
    ],
    mitre_tactics: [ 'TA0006' ],
    mitre_techniques: [ 'T1110.001' ],
    created_at: ISODate('2026-04-05T00:56:47.988Z'),
    updated_at: ISODate('2026-04-05T00:56:47.988Z')
  },
  {
    _id: ObjectId('69d1b34f27551dd738edcd6b'),
    incident_id: 'INC-2025-002',
    title: 'Malware détecté sur endpoint-pc-042',
    description: 'Trojan détecté avec suspicion d\'exfiltration',
    severity: 'critical',
    status: 'active',
    assigned_to: 'bob_senior',
  }
]

```

FIGURE 5 – Figure 5


```
soc_lite> docker exec -it mongo_secure mongosh -u soc_analyst_ro -p AnalystR0@2026! --authenticationDatabase s
soc_lite> db.security_logs.find().limit(2)

soc_lite> db.incidents.find({severity: "high"})
[
  {
    _id: ObjectId('69d1b34f27551dd738edcd6a'),
    incident_id: 'DKC-2025-001',
    title: 'Brute force SSH sur srv-web-01',
    description: 'Multiples tentatives SSH échouées depuis 192.168.1.105',
    severity: 'high',
    status: 'investigating',
    assigned_to: 'alice_analyst',
    tags: [ 'brute-force', 'ssh' ],
    related_log_ids: [
      ObjectId('69d1b34f27551dd738edcd64'),
      ObjectId('69d1b34f27551dd738edcd65')
    ],
    comments: [
      {
        author: 'alice_analyst',
        text: 'Pattern brute-force confirmé.',
        timestamp: ISODate('2025-03-03T10:35:00.000Z')
      }
    ],
    mitre_tactics: [ 'TA0006' ],
    mitre_techniques: [ 'T1110.001' ],
    created_at: ISODate('2026-04-05T00:56:47.988Z'),
    updated_at: ISODate('2026-04-05T00:56:47.988Z')
  }
]
```

FIGURE 6 – Figure 6